Informe Proyecto “CADENASPUNTEROS”

Alma Lazarte

🔬 Pregunta 1: Diferencia entre & y *

- **Explicación:** El operador ampersand (&edad) obtiene la ubicación física en la RAM, mientras que el asterisco (*p) accede al contenido de esa dirección.
- **Pregunta:** ¿Qué valor imprime en pantalla la instrucción cout << p; comparada con cout << *p;?
  - **Cout << p; imprime la dirección de memoria almacenada en el puntero**
  - **cout << *p; imprime el valor real de la variable a la que apunta el puntero**

🔬 Pregunta 2: Error Crítico al Guardar Entrada

- **Explicación:** Intentar escribir cin >> p; sin el asterisco es un error grave de compilación porque se intentaría sobrescribir la dirección de memoria física asignada al puntero.
- **Pregunta:** ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar cin para modificar el valor de la variable original?
  - **Es obligatorio usar *p para indicarle a cin que guarde la entrada en el valor de la variable apuntada. Sin el * intentariamos guardar el dato directo en la dirección de memoria del puntero, lo cual provoca un error de tipos al compilar**

🔬 Pregunta 3: Formato Hexadecimal

- **Explicación:** Las direcciones de memoria RAM se muestran en formato hexadecimal (base 16) antecedidas por 0x (ej: 0x61fe1c).
- **Pregunta:** ¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?
  - **Se usa hexadecimal porque cada dígito representa exactamente 4 bits en . binario. Esto permite condensar direcciones de memoria muy largas en pocos caracteres y facilitar la lectura técnica del hardware.**

Link github: https://github.com/almaaluzz05/CADENASPUNTEROS.git